

Status_Summary

CodeGraph Status — Boba — Status Summary

Revision: 1 **Last modified:** 2026-08-20T12:27:23Z **Companion of:** docs/codegraph/Status.md (Rev 2, §11.4.56 two-audience summary). **Scope:** Two-audience summary of the CodeGraph install/update/sync/validate ledger for the Boba project (§11.4.78/§11.4.79/§11.4.80).

Every claim here traces to Status.md Rev 2. No overclaim, no invented PASS (§11.4.6). **New this pass (BOB-083 / GOVERNANCE_AUDIT_2026-08-08_ROUND2.md RD2-16):** this summary did not previously exist — docs/codegraph/Status.md had no Status_Summary companion, a real §11.4.56 gap. Created now; Status.md itself was left unchanged (its own Rev 2 content, 2026-08-18, was independently re-checked this session — see “Re-confirmed today” below — and found still accurate, so no new Status.md revision was warranted).

Page 1 — For the team (plain language)

CodeGraph is a developer tool that reads through the whole Boba codebase and builds a searchable map of it (which function calls which, which file defines what), so an AI coding assistant can look things up instantly instead of reading every file by hand. It is a productivity aid, not something end users of Boba ever see or depend on.

What works today:

- CodeGraph is installed on this development machine (version 1.5.0) and was successfully set up and validated once, back on 2026-06-06 — at that time it produced a working map of the project

(509 files, ~8,900 code entries) and every safety check passed: no secrets got indexed, no third-party vendor code got indexed, the project's own shared governance code got indexed correctly.

What is currently NOT working, and why (honestly, not glossed over):

- **The map is out of date and not currently usable.** The last time it was successfully rebuilt was 2026-06-06. On 2026-08-18, a rebuild was attempted and it went badly: the newer version of the tool has a bug where it doesn't correctly skip a couple of large, irrelevant folders (frontend/node_modules and extension/node_modules — third-party library code, not Boba's own code) the way it should, so instead of mapping ~500 files it started trying to map over 32,000 files and ballooned to a 1.8 GB database before being deliberately stopped to avoid wasting machine resources on a documentation task. Nothing broke — the half-finished attempt was safely thrown away, and the map is simply back to “not currently built” rather than corrupted or half-broken.
- **Re-confirmed today (2026-08-20, this pass):** a quick, safe, read-only check (asking the tool for its current status — no rebuild attempted) confirms the same thing is still true two days later: the tool is present (still version 1.5.0), and the map is still not built (the status check itself fails with a “no such table” error, exactly as expected for a not-yet-built map). This is not a new problem — it is the same, already-known, already-honestly-reported situation, just double-checked.

Team / operator actions: none required urgently — CodeGraph not being live does not block any Boba feature from working; it only means the AI coding assistant falls back to reading files directly (slower, but functionally fine). Whoever picks up rebuilding the map should first fix the tool's known “skips large nested library folders” bug (or add an explicit exclusion for the two known-offending folders), rather than just re-running the same command and hitting the same 1.8 GB blowup again.

Page 2 — For software engineers

Tool: codegraph CLI, npm package @colbymchenry/codegraph. **Installed version (re-checked live, 2026-08-20):** 1.5.0 (codegraph --version), matches Status.md's 2026-08-18 entry — no version drift in the intervening 2 days.

Live index state (re-checked, read-only, 2026-08-20): codegraph status returns `x Failed to get status: no such table: unresolved_refs`; `.codegraph/codegraph.db` is 4 KB (empty schema, not a built index). This is the SAME state Status.md's 2026-08-18 entry documented — **not re-indexed by this pass** (deliberately: a fresh `codegraph init` on this tree is what produced the documented 32,260-file / 1.8 GB blowup two days ago; re-attempting it inside a bounded docs-sync task would repeat the same host-resource-discipline violation §12.6/§12.11 already flagged, so this pass is strictly observational).

Timeline (mirrors Status.md)

Date	Event	Result
2026-06-06T14:30:00Z	Initial install (0.9.7→0.9.9) + first index	509 files / 8,906 nodes / 17,025 edges; codegraph_validate.sh 7 PASS / 0 FAIL
2026-06-06T14:38:30Z	codegraph_sync.sh run	baseline == post-sync (2,599 methods / 1,722 functions / 498 files / 139 constants / 85 structs / 84 interfaces); validate PASS Blocked by a real, confirmed CodeGraph 1.5.0 regression: nested (non-root) .gitignore files (frontend/.gitignore, extension/.gitignore) are no longer honored, so codegraph init walked into both node_modules trees (365 MB + 236 MB) despite
2026-08-18T13:33:41Z	BOB-075 staleness remediation — re-index attempted	

Date	Event	Result
2026-08-20T12:27:23Z	This pass (BOB-083) — read-only re-confirmation	git check-ignore -v proving git itself correctly excludes them. Run aborted at 32,260 files / 514,456 nodes / 724,013 edges / 1.8 GB db (§12.6/§12.11 host-resource discipline); truncated db removed (gitignored, never tracked) codegraph --version → 1.5.0 (unchanged); codegraph status → same “no such table” failure (unchanged); no re-index attempted — this pass only creates the missing Status_Summary companion

Root cause of the current not-live state (unresolved, tracked as an honest gap)

Status.md’s 2026-08-18 entry root-caused this as CONFIRMED (not guessed, §11.4.6): a CodeGraph 1.5.0 regression in nested-.gitignore handling. The 2026-06-06 baseline (509 files) existed BEFORE frontend/ and extension/ (each with their own node_modules + nested .gitignore) were added to this project, so the zero-config “exclusion driven by .gitignore” behavior documented at install time was never exercised against nested .gitignore files before the 2026-08-18 attempt surfaced the gap. **Not fixed by either the 2026-08-18 or this 2026-08-20 pass** — the documented remediation path is either (a) investigate/report the CodeGraph 1.5.0 regression upstream, or (b) add an explicit frontend/ node_modules/ + extension/node_modules/ exclusion at this project’s own root .gitignore (the root file IS honored per the 2026-06-06 baseline’s own finding) — neither was performed in either pass; no

new tracked workable item was minted for the CodeGraph-side regression itself (BOB-072/073 owns broader DB work; this stays an in-repo honest flag per §11.4.6 rather than a silent defer).

Key paths

- Ledger source: docs/codegraph/Status.md (append-only; new entries appended by constitution/scripts/codegraph_sync.sh).
- Sync automation: constitution/scripts/codegraph_sync.sh, codegraph_update.sh, codegraph_validate.sh (§11.4.80).
- Exclusions: root .gitignore (.codegraph/codegraph.db*, submodules/jackett/, secret paths per §11.4.10).
- MCP wiring: .mcp.json registers codegraph serve --mcp (stdio) — present and reachable in this session (the codegraph_explore MCP tool responded), independent of whether the on-disk index is currently built.

Anti-bluff note (§11.4.6)

Both the 2026-08-18 blocked-reindex finding and this pass's "still not live, still version 1.5.0" re-confirmation are from REAL command output captured in-session (codegraph --version, codegraph status, du -sh .codegraph/codegraph.db), never assumed carried-forward. No index rebuild was attempted or claimed successful by this pass — the honest state is "not live," stated plainly, not obscured by omission.